

VIETNAM NATIONAL UNIVERSITY - HO CHI MINH
CITY INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



**WEB APPLICATION DEVELOPMENT
REPORT FOR PROJECT
BOOKSTORE
GROUP 11 - MEMBER LIST**

- | | |
|--------------------|-------------|
| 1. Nguyễn Thái Bảo | ITCSIU24013 |
| 2. Trần Khánh Bình | ITCSIU24015 |

Instructor: Nguyễn Trung Nghĩa

Submission Date: 31/05/2026

Repository: Bookstore Project Web

Deployment URL: <https://boned-gerbil-tattle.ngrok-free.dev>

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION

- 1.1. Abstract
- 1.2. Background Research and System Overview
- 1.3. Project Goals
- 1.4. Techniques and Tools Used
- 1.5. Scope and Limitations
- 1.6. Structure of the Report

CHAPTER 2 REQUIREMENTS ANALYSIS AND TASK TIMELINE

- 2.1. Target Users and Pain Points
- 2.2. User Stories
- 2.3. Functional Requirements
- 2.4. Non-Functional Requirements
- 2.5. Team Allocation and Individual Contributions

CHAPTER 3 METHODOLOGY

- 3.1. System Architecture Overview
- 3.2. Technology Stack Justification
- 3.3. Design Patterns
- 3.4. Entity Relationship Diagram
- 3.5. Database Creation and Normalization
- 3.6. Application Structure and Class Structure
- 3.7. API and Route Design
- 3.8. Key Business Logic and Security
- 3.9. Advanced Features and AI Assistance

CHAPTER 4 IMPLEMENTATION, TESTING AND DEPLOYMENT

- 4.1. Implementation Process
- 4.2. User Interface and Feature Walkthrough
- 4.3. Testing and Quality Assurance
- 4.4. Bug Tracking
- 4.5. Deployment Process
- 4.6. Results and Analysis
- 4.7. Challenges and Solutions

CHAPTER 5 CONCLUSION AND FUTURE WORK

- 5.1. Summary of Key Findings
- 5.2. Reflection on Objectives
- 5.3. Future Work and Recommendations

REFERENCES

APPENDICES

LIST OF FIGURES

Figure 3.1: High-Level System Architecture

Figure 3.2: Checkout and Reservation Data Flow

Figure 3.3: ERD of IU Bookstore Management System

Figure 4.1: Registration Interfaces

Figure 4.2: Book Catalog Interfaces

Figure 4.3: Manager Dashboard

LIST OF TABLES

Table 1.1: Project Summary

Table 1.2: Summary of Technologies and Development Tools

Table 2.1: Target Users and Pain Points

Table 2.2: Prioritized User Stories

Table 2.3: Summary of Functional Requirements and Current Implementation

Table 2.4: Summary of Non-Functional Requirements and Current Design

Table 3.1: Layered Architecture Responsibilities

Table 3.2: Normal Form Achievement

Table 3.5.1: Creation Summary

Table 3.5.2. Database Tables Summary

Table 3.5.3. Database Tables Summary

Table 4.1: Summary of Requirement Implementation Status

Table 4.2: Testing Matrix

Table 4.3: Deployment Environment Variables

Table 4.4: Test Accounts

CHAPTER 1 INTRODUCTION

1.1. Abstract

The IU Bookstore Management System is a full-stack web application developed for the Web Application Development course. The system supports an online university bookstore in which customers can browse books, search and filter the catalog, place orders, reserve books, manage their profiles, and receive notifications. Managers can maintain catalog data and process operational records, while staff users support order and reservation follow-up according to the access rules configured in Spring Security. Administrators can manage users and audit logs.

The project was implemented with Spring Boot 3.3.5, Java 17, Thymeleaf, Spring Security, Spring JDBC/JdbcTemplate, MySQL, H2 for local demo support, and Chart.js for dashboard visualization. The uploaded repository contains 36 Java source files, 21 Thymeleaf templates, 25 database tables, 178 table columns, 8 explicit indexes, and 87 commits on the current working branch. These figures show that the project is not a simple CRUD prototype but a structured web application with authentication, role-based authorization, catalog management, ordering, reservation, notification, audit, and reporting features.

Category	Current Implementation
Project domain	Online university bookstore management system
Core users	Admin, Manager, Staff, Customer
Main features	Registration, login, profile, catalog, cart, checkout, orders, reservations, notifications, dashboard, CSV export
Advanced features	Dashboard charts, CSV export, in-app notifications, audit log, reservation workflow
Database scale	25 tables, 178 columns, 8 indexes, multiple foreign key relationships
Repository evidence	More than 80 commits detected in Git history
Current risk	Automated test evidence and production-level deployment hardening should be improved before final submission.

Table 1.1: Project Summary

1.2. Background Research and System Overview

A bookstore system is often underestimated as a simple list of products. In practice, a bookstore must coordinate product information, authorship, inventory, pricing, discount rules, customer accounts, carts, order items, payment records, shipping records, reservations, user roles, and operational reports. If these activities are handled manually or through disconnected spreadsheets, the store can oversell unavailable books, lose order history, fail to inform customers about reservation status, and make it difficult for managers to monitor stock and service performance.

The proposed system addresses these problems by combining a normalized relational database with a Spring Boot MVC application. Unlike a static catalog page, the system controls access according to user roles. Customers have routes for browsing, cart, checkout, reservations, orders, profile, and notifications. Managers have access to catalog management, coupons, orders, reservations, reports, and CSV export. Staff users support operational workflows such as order and reservation follow-up

according to the final Spring Security configuration. This role-aware structure is important because it prevents sensitive management functions from appearing to ordinary customers.

The system uses a server-rendered frontend approach. Thymeleaf templates are stored under `src/main/resources/templates`, and CSS, JavaScript, and images are stored under `src/main/resources/static`. Therefore, the frontend is integrated into the Spring Boot backend module rather than separated into a React or Vue project. This is a valid hybrid approach because the initial page is rendered by the server while JavaScript is used for interactive behavior such as dashboard charts and UI assistance.

1.3. Project Goals

- Develop a complete web application that integrates frontend pages, backend controllers, business services, repository classes, and a relational database.
- Implement secure user authentication, BCrypt password hashing, and role-based access control for at least three distinct user roles.
- Provide meaningful CRUD operations for main entities such as books, categories, authors, coupons, orders, reservations, and users.
- Apply database design principles including primary keys, foreign keys, junction tables, constraints, audit columns, indexing, and Third Normal Form.
- Demonstrate domain-specific business logic such as stock checking, coupon validation, order total calculation, soft deletion, reservation workflow, and notification handling.
- Deliver advanced features beyond CRUD, especially dashboard charting, CSV export, audit logging, and notification center behavior.
- Use Git and GitHub as version-control tools with meaningful commits and branch-based collaboration.
- Prepare a technical report and live demonstration that can be understood by both technical and non-technical audiences.

1.4. Techniques and Tools Used

Layer / Purpose	Technology	Use in project
Frontend	Thymeleaf, HTML, CSS, JavaScript, Chart.js	Server-rendered pages, responsive layout, form pages, dashboard charts
Backend	Spring Boot 3.3.5, Java 17	Controllers, service logic, validation, routing, application configuration
Security	Spring Security, BCrypt	Login/logout, role-based access, password hashing, CSRF protection
Database Access	Spring JDBC / JdbcTemplate	Parameterized SQL queries and repository layer
Database	MySQL, H2	Production-like relational storage and optional local/demo database support
Version Control	Git, GitHub	Branching, commit history, collaboration, repository audit

Build Tool	Maven	Dependency management and application packaging
Deployment Support	Docker Compose	MySQL container and repeatable local environment setup

Table 1.2: Summary of Technologies and Development Tools

1.5. Scope and Limitations

The application scope includes user registration and login, catalog browsing, cart management, order placement, reservation submission, profile management, notifications, manager catalog tools, order and reservation management, CSV export, dashboard metrics, administrator user control, and audit log review. The system is designed for one university bookstore rather than a multi-branch national retail platform.

The current repository already demonstrates a strong database schema and a complete server-rendered MVC flow. However, before final submission, the team should verify that the public deployment URL remains accessible during the demonstration, improve the README, remove hardcoded credentials from application-mysql.properties by replacing them with environment variables, and add automated tests under src/test. These limitations are documented honestly so the team can complete the final polishing work before presentation.

1.6. Structure of the Report

Chapter 2 analyzes requirements, user stories, non-functional requirements, team allocation, and timeline. Chapter 3 presents the methodology, including architecture, technology justification, database design, normalization, route design, business logic, security, advanced features, and AI assistance. Chapter 4 explains implementation, user interface walkthrough, testing, deployment, results, and challenges. Chapter 5 concludes the project and proposes future improvements. The appendices contain route references, test accounts, database table summaries, Git evidence, and a compliance checklist.

CHAPTER 2 REQUIREMENTS ANALYSIS AND TASK TIMELINE

2.1. Target Users and Pain Points

Role	Needs	Pain points solved by the system
Customer	Browse books, search/filter catalog, manage profile, add to cart, checkout, reserve books, track orders, receive notifications	Avoid repeated manual communication, receive clear service status, and prevent purchasing unavailable books
Staff	Assist order processing and reservation follow-up	Reduce repetitive manual updates and avoid operational confusion
Manager	Manage books, categories, authors, coupons, orders, reservations, dashboard reports, and CSV export	Gain operational visibility into stock, orders, and service workflow
Admin	Manage users, roles, and audit logs	Centralize access control and accountability across the application

Table 2.1: Target Users and Pain Points

2.2. User Stories

Priority	User Story
Must Have	As a customer, I want to register an account so that I can order and track books.
Must Have	As a customer, I want to log in and log out securely so that my personal data is protected.
Must Have	As a customer, I want to search books by keyword so that I can quickly find a target book.
Must Have	As a customer, I want to filter books by category, price, status, and stock so that I can narrow results.
Must Have	As a customer, I want to add books to a cart so that I can prepare an order.
Must Have	As a customer, I want the system to prevent checkout when stock is insufficient so that I do not pay for unavailable items.
Must Have	As a customer, I want to reserve a book so that staff can prepare or follow up when the book is available.
Must Have	As a customer, I want to receive notifications so that I know order or reservation updates.
Must Have	As a manager, I want to create and update books so that the catalog remains accurate.
Must Have	As a manager, I want to soft delete books so that old order history remains valid.

Must Have	As a manager, I want to update order status so that customers can follow processing progress.
Must Have	As an admin, I want to create and manage users so that account access can be controlled without directly editing the database.
Must Have	As an admin, I want to inspect audit logs so that important actions are accountable.
Should Have	As a manager, I want dashboard charts so that system performance can be monitored visually.
Should Have	As an admin, I want unread notifications so that important updates are not missed.
Should Have	As a customer, I want to edit my profile so that my contact information stays up to date.
Should Have	As a manager, I want to export book data to CSV so that reports can be shared outside the application.
Nice to Have	As a customer, I want VNPay or online payment integration so that checkout becomes more convenient.
Nice to Have	As a manager, I want reorder forecasting so that stock planning becomes more proactive.

Table 2.2: Prioritized User Stories

2.3. Functional Requirements

Requirement	Target	Current implementation
User management	Registration, login/logout, profile, password change, at least 3 roles, admin management	Implemented with registration, login, logout, profile, password management, Spring Security, BCrypt password hashing, and four roles: ADMIN, MANAGER, STAFF, CUSTOMER. Email verification is implemented through an email verification token workflow: after registration, the system generates a verification token, sends a verification link to the user's email address, and updates the email_verified status after the user confirms the account. Relational schema is strong; externalized configuration and permanent cloud deployment are recommended for production readiness.

Role and RBAC	Different roles see different features and data	Implemented by Spring Security and route separation for /admin, /manager, customer routes, and role-aware navigation.
CRUD operations	At least 4 main entities with create, read, update, delete	Implemented for books, categories, authors, coupons, user management, order status, and reservation status. Books use soft delete through status.
Search/filter/pagination	Multi-criteria search, filters, sorting, page size	Implemented mainly for catalog and manager book listing with keyword, category, status, price/stock conditions, sorting, and PageResult support.
Business logic	Domain-specific rules beyond simple CRUD	Implemented through stock validation, coupon validation, order creation, payment/shipment records, inventory transactions, reservation workflow, notifications, and audit logs.
Advanced features	Minimum 3 advanced features	Dashboard with Chart.js, CSV export for books, notification center, audit log, reservation workflow.

Table 2.3: Summary of Functional Requirements and Current Implementation

2.4. Non-Functional Requirements

Requirement	Target	Current design
Performance	Common pages respond within 1-2 seconds for classroom-scale data	Indexes exist for common query columns; server-side rendering reduces frontend build overhead.
Security	Hashed password, RBAC, CSRF, no SQL injection	BCrypt and RBAC are implemented. JdbcTemplate uses parameterized queries for most database operations. CSRF protection is enabled through Spring Security.
Usability	Responsive layout, clear messages, confirmations	CSS templates include navigation, forms, error messages, and confirmation flows for destructive actions.
Reliability	No critical bugs during demo	Manual testing should be completed before presentation;

		automated tests are a current improvement point.
Maintainability	Layered code, meaningful names, documentation	MVC packages exist: controller, service, repository, model, dto, config. README must be expanded before final submission.
Scalability	Support realistic sample data and future deployment	Relational schema is strong; externalized configuration and permanent cloud deployment are recommended for production readiness.

Table 2.4: Summary of Non-Functional Requirements and Current Design

2.5. Team Allocation and Individual Contributions

Area	Responsible member(s)	Main responsibilities
Project leadership and integration	Nguyễn Thái Bảo, Trần Khánh Bình	Coordinate work, database inspection, report preparation, Git workflow, final integration.
UI and feature development	Trần Khánh Bình	Improve interface, customer pages, reservation display, role-based UI navigation.
Backend and security	Nguyễn Thái Bảo, Trần Khánh Bình	Controllers, Spring Security configuration, service logic, repositories.
Database design	Nguyễn Thái Bảo Trần Khánh Bình	Schema design, constraints, indexes, seeding strategy, ERD maintenance.
Testing and documentation	Nguyễn Thái Bảo	Manual test cases, report, README, deployment notes, API documentation.

More than 80 commits detected in Git history, with major contributions from Nguyen Thai Bao and Tran Khanh Binh. The official submitted member list should be edited on the cover page if the class submission includes additional members.

CHAPTER 3 METHODOLOGY

3.1. System Architecture Overview

The application follows a layered, server-rendered MVC architecture. Browser requests pass through the Spring Security filter chain before reaching Spring MVC controllers. Controllers collect request parameters and delegate business workflows to services or direct data operations to repository classes. Repositories use JdbcTemplate to query MySQL or H2. Results are returned to controllers, placed into a Model, and rendered through Thymeleaf templates. Static CSS and JavaScript support layout, client-side behavior, and dashboard charting.

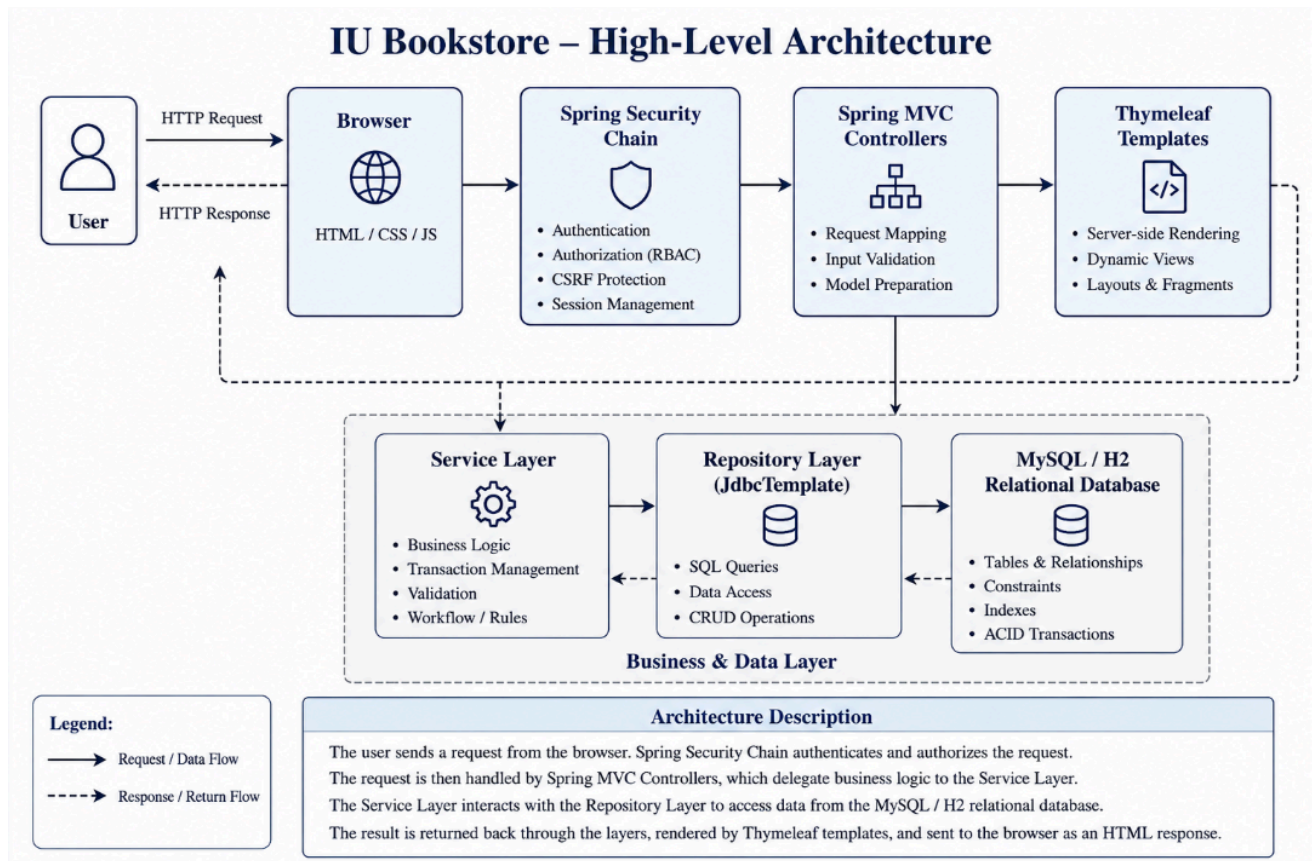


Figure 3.1: High-Level System Architecture

Layer	Main files/folders	Responsibilities
Presentation	src/main/resources/templates, static/css/app.css, static/js/app.js	Render pages, forms, navigation, catalog, dashboard charts, validation helpers.
Controller	src/main/java/com/controller	Handle routes such as /catalog, /cart, /manager/books, /admin/users.
Service	src/main/java/com/service	Business workflows, current-user lookup, checkout, discount calculation.
Repository	src/main/java/com/repository	SQL queries and database updates through JdbcTemplate.
Model/DTO	src/main/java/com/model and com/dto	Book, Category, Author, Coupon, AppUser, Reservation, OrderSummary, CartLine, PageResult.
Database	database/schema.sql and DatabaseSeeder.java	Tables, constraints, indexes, initial demo data.

Table 3.1: Layered Architecture Responsibilities

High-Level Data Flow of the IU Bookstore System

User → Browser → Spring Security Chain → Controller → Service/Repository → MySQL/H2 Database → Repository Result → Controller Model → Thymeleaf Template → HTML Response

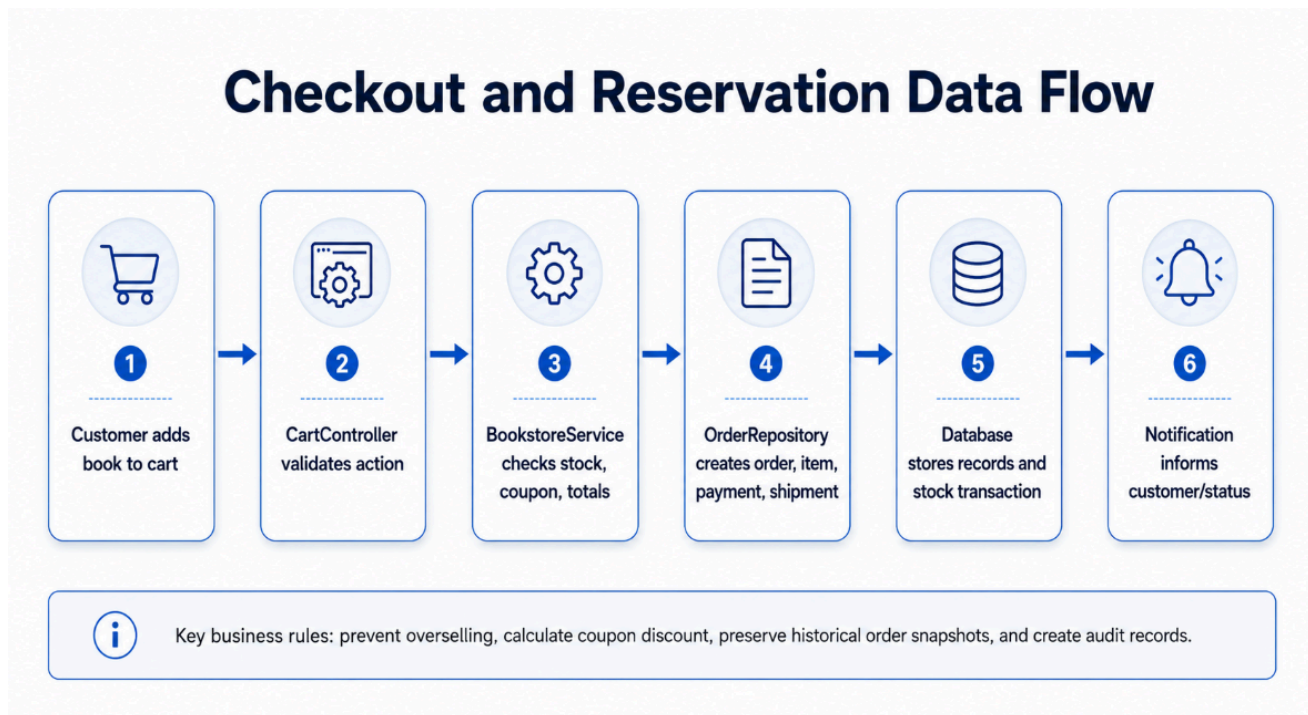


Figure 3.2: Checkout and Reservation Data Flow

3.2. Technology Stack Justification

Spring Boot was chosen over raw Java Servlets because the project requires authentication, authorization, validation, database integration, structured routing, dependency management, and fast development within five weeks. Raw Servlets would demonstrate fundamentals but would require significantly more boilerplate for routing, session security, CSRF handling, and MVC structure. Spring Boot provides a production-like foundation while still allowing the team to understand controller, service, repository, and configuration responsibilities.

Thymeleaf was chosen over a standalone React/Vue frontend because the application is mostly form-driven and server-rendered pages are sufficient for the required workflows. A separate SPA would require API design, CORS configuration, token storage, frontend routing, and a build pipeline. Thymeleaf keeps frontend and backend integration simple, while JavaScript and Chart.js still support interactive dashboard behavior. This hybrid approach matches the project requirement allowing JSP/Thymeleaf with JavaScript interactivity.

MySQL was selected as the main relational database because it is widely used, easy to install locally, and suitable for the relational nature of bookstore data. The schema contains many foreign key relationships and junction tables, which are easier to enforce in SQL than in a document database. H2 is included only for local demo or testing purposes; the production-like configuration uses MySQL.

3.3. Design Patterns

- MVC Pattern: Controllers receive requests, models represent data, and Thymeleaf views render HTML responses.
- Layered Architecture: Presentation, controller, service, repository, model/DTO, and database layers separate responsibilities.
- Repository Pattern: SQL access is centralized in repository classes instead of being mixed into controllers or templates.
- Service Layer Pattern: BookstoreService and CurrentUserService encapsulate business workflows and current-user logic.
- DTO Pattern: CartLine and PageResult carry view-specific data without exposing raw database rows directly.
- Configuration Pattern: SecurityConfig and DatabaseSeeder centralize application-level setup and initial data generation.

3.4. Entity Relationship Diagram (ERD)

The database model is designed to support the core functions of the IU Bookstore Management System, including user account management, role assignment, address management, book catalog organization, shopping cart operations, wishlist management, book reservations, order processing, payment tracking, coupon usage, and customer reviews. Based on the final schema.sql implementation, the database contains 25 tables covering user management, roles and permissions, addresses, catalog data, carts, wishlists, reservations, orders, payments, shipments, inventory transactions, reviews, notifications, and audit logs. The schema includes core entities, supporting entities, transaction entities, and junction tables used to represent many-to-many relationships.

The database includes several one-to-many relationships. For example, one user can have multiple addresses, cart items, wishlist items, reservations, orders, and reviews. One order can contain multiple order items, and one book can appear in many cart items, wishlist items, reservation records, order items, and reviews. These relationships allow the system to store customer activities and transaction records in a structured way.

The schema also contains several many-to-many relationships, which are implemented through junction tables. The relationship between app_user and role is handled through user_role, allowing one user to have multiple roles and one role to be assigned to many users. The relationship between books and authors is handled through book_authors, allowing one book to have multiple authors and one author to write multiple books. In the current implementation, each book belongs to one primary category through the category_id foreign key in the book table. This represents a many-to-one relationship from book to category. A future version may introduce a book_category junction table if the system needs to support books belonging to multiple categories.

Overall, the ERD follows a normalized relational structure. Core entities such as users, books, orders, authors, categories, and roles are separated into independent tables, while junction tables are used to avoid data duplication in many-to-many relationships. Transaction-related data is also separated clearly: order information is stored in orders, purchased items are stored in order_items, and payment details are stored in payments. This structure improves data consistency, supports future maintenance, and makes the database suitable for a university-level bookstore management web application.

3.5. Database Creation and Normalization

The database schema is created by database/schema.sql. Tables are dropped and created in dependency order so that foreign key constraints can be applied correctly. Independent tables such as users, roles, books, authors, categories, and coupons are created before dependent tables such as user_roles, addresses, book_authors, book_categories, cart_items, wishlist_items, reservations, orders, order_items, payments, and reviews.

The schema is designed to support the main features of the IU Bookstore Management System, including user account management, role assignment, book catalog management, book classification, shopping cart, wishlist, reservation, order processing, payment tracking, customer reviews, coupon application, and address management. The ERD uses primary keys and foreign keys to maintain referential integrity between related tables.

Metric	Value
Number of tables	25
Total columns	178
Explicit indexes	8
Main junction tables	user_role, role_permission, book_author, cart_item, wishlist_item
Relationship-like item tables	cart_item, wishlist_item, order_item, inventory_transaction
Sample data source	DatabaseSeeder.java; seed-data.sql currently documents that demo data is generated programmatically

Table 3.5.1: Creation Summary

No.	Table	Main Columns / Attributes	Role in Schema	Detailed Description
1	app_user	user_id, username, email, password_hash, full_name, phone, status, created_at, updated_at	Core entity	Stores the main account information of users in the system. This table is used for login, customer identity, profile management, and linking users to orders, carts, reservations, reviews, notifications, and audit logs.
2	role	role_id, role_name, description	Security entity	Stores system roles such as ADMIN, MANAGER, STAFF, or CUSTOMER. Roles are separated from users to support role-based access control.
3	user_role	user_id, role_id, created_at	Junction table	Resolves the many-to-many relationship between app_user and role. One user may have multiple roles, and one role may belong to many users.
4	permission	permission_id, permission_name, description	Security entity	Stores detailed permissions that can be assigned to roles. This supports future expansion if the system needs more detailed authorization beyond basic roles.

5	role_permission	role_id, permission_id, created_at	Junction table	Resolves the many-to-many relationship between role and permission. It allows each role to contain multiple permissions.
6	address	address_id, user_id, recipient_name, phone, street, ward, district, city, country, is_default, created_at	Supporting entity	Stores user delivery or contact addresses. A user may have multiple addresses, and one address can be selected during checkout.
7	category	category_id, category_name, slug, description, parent_id, status, created_at	Catalog entity	Stores book category information. Categories help users browse and filter books by topic or genre. If parent_id is used, the system can support parent-child category structures.
8	author	author_id, author_name, biography, nationality, birth_date, status, created_at, updated_at	Catalog entity	Stores author information. Authors are separated from books to avoid repeating author details when one author writes many books.
9	book	book_id, isbn, title, description, publisher, publication_year, price, stock_quantity, cover_image, status, created_at, updated_at	Core catalog entity	Stores the main information about books. This is one of the central tables of the system because it is connected to authors, categories, cart items, wishlist items, reservations, order items, reviews, and inventory transactions.

10	book_author	book_id, author_id, created_at	Junction table	Resolves the many-to-many relationship between book and author. One book may have multiple authors, and one author may write many books.
11	coupon	coupon_id, code, discount_type, discount_value, minimum_order_amount, start_date, end_date, usage_limit, status	Promotion entity	Stores coupon codes that customers can apply during checkout. It supports discount validation, minimum order rules, date limits, and usage restrictions.
12	promotion	promotion_id, promotion_name, description, discount_type, discount_value, start_date, end_date, status	Promotion entity	Stores promotional campaigns. This table supports marketing features such as seasonal discounts or special sales events.

Table 3.5.2. Database Tables Summary

Number	Table	Main Columns / Attributes	Role in Schema	Detailed Description
13	cart	cart_id, user_id, cart_status, created_at, updated_at	Transaction support entity	Stores the shopping cart header of a user. It separates the cart itself from the items inside the cart, making the design cleaner and easier to extend.

14	cart_item	cart_item_id, cart_id, book_id, quantity, unit_price, added_at	Transaction detail entity	Stores individual books added to a cart. Each record represents one book in one cart, including quantity and price-related information.
15	wishlist	wishlist_id, user_id, wishlist_name, created_at, updated_at	Supporting entity	Stores wishlist header information. This allows a user to save books for future purchase.
16	wishlist_item	wishlist_item_id, wishlist_id, book_id, added_at	Supporting detail entity	Stores books saved inside a wishlist. It prevents storing multiple book IDs directly in the wishlist table.
17	reservation	reservation_id, user_id, book_id, quantity, status, reserved_at, expires_at, updated_at	Core transaction entity	Stores book reservation requests. This table is used when customers reserve books that may be unavailable, limited in stock, or waiting for staff approval.
18	customer_order	order_id, user_id, address_id, coupon_id, order_status, subtotal, discount_amount, total_amount, payment_status, order_date, created_at, updated_at	Core transaction entity	Stores order header information. It records who placed the order, which address was used, the payment/order status, and total

				payment calculation.
19	order_item	order_item_id, order_id, book_id, quantity, unit_price, discount_amount, line_total, title_snapshot	Transaction detail entity	Stores books inside an order. Snapshot fields such as unit_price and title_snapshot preserve historical accuracy even if book information changes later.
20	payment	payment_id, order_id, payment_method, payment_status, amount, transaction_reference, paid_at, created_at	Core transaction entity	Stores payment information for orders. One order may have one payment record or multiple payment attempts depending on the payment process.
21	shipment	shipment_id, order_id, shipment_status, carrier, tracking_number, shipped_at, delivered_at, created_at	Operational entity	Stores delivery and shipping information. It helps managers and customers track order delivery status.
22	inventory_transaction	transaction_id, book_id, order_id, transaction_type, quantity_change, stock_after, reason, created_at, created_by	Tracking entity	Stores stock movement history. Instead of only updating the stock quantity in book , this table records why and when stock changed.

23	review	review_id, user_id, book_id, rating, comment, status, created_at	Supporting entity	Stores customer ratings and comments for books. It supports product feedback and helps other users evaluate books.
24	notification	notification_id, user_id, reservation_id, order_id, title, message, notification_type, is_read, read_at, created_at	Supporting entity	Stores notifications shown to users. Notifications can be triggered by reservation updates, order updates, or system messages.
25	audit_log	audit_id, actor_user_id, entity_name, entity_id, action, old_value, new_value, created_at	Tracking entity	Stores important system actions for accountability. This is useful for admin monitoring, debugging, and explaining who changed what in the system.

Table 3.5.3. Database Tables Summary

3.5.1. Functional Dependencies and 3NF Analysis

The database schema was designed to satisfy Third Normal Form. Each table stores facts about one main entity or relationship, and non-key attributes depend on the primary key rather than on other non-key attributes.

For example, in `app_user`, `user_id` determines `email`, `password_hash`, `full_name`, `phone`, `status`, `email_verified`, `created_at`, and `updated_at`. The `email` column is also unique, which prevents duplicate accounts. In `book`, `book_id` determines `isbn13`, `sku`, `title`, `description`, `publisher`, `publication_year`, `price`, `stock_on_hand`, `reorder_level`, `status`, and audit timestamps. Author information is not stored repeatedly inside the `book` table; instead, the many-to-many relationship is represented through `book_author`.

In `customer_order`, `order_id` determines `user_id`, `address_id`, `coupon_id`, `order_status`, `subtotal`, `discount_amount`, `total_amount`, `payment_status`, `order_date`, `created_at`, and `updated_at`. Individual purchased books are not stored as repeating columns inside `customer_order`. They are stored in `order_item`, where each row represents one book line in one order. This design avoids repeating groups and supports accurate order history.

The schema avoids transitive dependency by separating roles, permissions, addresses, authors, categories, payments, shipments, reviews, notifications, and audit logs into independent tables. Therefore, the design satisfies 1NF through atomic attributes, 2NF through full dependency on primary keys, and 3NF by removing transitive dependencies between non-key attributes.

3.5.2. Indexing Strategy

The schema defines indexes to improve the performance of common database operations in the IU Bookstore Management System. These indexes mainly support user login, catalog browsing, book searching, cart management, wishlist management, reservation tracking, order history, payment lookup, and review display.

Indexes should be created on frequently searched columns and foreign key columns. For example, user-related indexes help the system quickly find accounts during login and registration. Book-related indexes support searching and filtering books by title, status, price, and stock availability. Order-related indexes help customers and managers retrieve order history efficiently. Reservation and review indexes help the system display user requests and book feedback more effectively.

Indexed Area	Suggested Indexed Columns	Purpose
User account lookup	<code>users.email</code> , <code>users.username</code>	Supports login, registration validation, and duplicate account checking.
User-role lookup	<code>user_roles.user_id</code> , <code>user_roles.role_id</code>	Supports role-based access control after a user logs in.
Address lookup	<code>addresses.user_id</code>	Supports retrieving delivery addresses of a user.

Book search and filtering	books.title, books.status, books.price, books.stock_quantity	Supports catalog browsing, searching, sorting, and stock checking.
Book-author relationship	book_authors.book_id, book_authors.author_id	Supports loading authors of a book and books written by an author.
Cart lookup	cart_items.user_id, cart_items.book_id	Supports displaying and updating the current user's cart.
Wishlist lookup	wishlist_items.user_id, wishlist_items.book_id	Supports displaying books saved by a user.
Reservation lookup	reservations.user_id, reservations.book_id, reservations.status	Supports customer reservation history and manager reservation filtering.
Order lookup	orders.user_id, orders.order_status, orders.order_date	Supports customer order history and manager order management.
Order detail lookup	order_items.order_id, order_items.book_id	Supports displaying items inside an order.
Payment lookup	payments.order_id, payments.payment_status	Supports payment tracking for each order.
Review lookup	reviews.book_id, reviews.user_id, reviews.status	Supports displaying reviews on book detail pages and checking user review history.
Coupon lookup	coupons.code, coupons.status	Supports coupon validation during checkout.

Foreign key columns are important indexing targets because they are frequently used in join operations. For example, `order_items.order_id` is used to load all items of an order, while `book_author.book_id` and `book.category_id` are used to display detailed book and category information. Proper indexing helps the system remain efficient when the number of users, books, orders, and reservations increases..

3.6. Application Structure and Class Structure

The source code follows a clear package structure based on the separation of concerns principle. Each package has a specific responsibility, which makes the project easier to understand, maintain, debug, and extend.

The com.controller package contains route handlers that receive HTTP requests, call the required services, and return the appropriate Thymeleaf pages. The com.service package contains business services that coordinate important workflows such as checkout, user lookup, cart processing, and reservation handling. The com.repository package contains SQL data access logic and communicates directly with the database by using JdbcTemplate. The com.model package contains domain objects that represent the main business entities of the system. The com.dto package contains data transfer objects used to prepare data for views and pagination. The com.config package contains configuration classes for security, password encoding, and initial data setup.

This structure improves maintainability because SQL logic, business logic, page routing, and configuration are not mixed into one file. It also makes the project more suitable for teamwork because different members can work on different layers of the application without frequently modifying the same files.

Package	Representative Files	Responsibility
com.config	SecurityConfig, DatabaseSeeder	Contains security rules, password encoder configuration, and initial data generation.
com.controller	AuthController, CatalogController, CartController, ManagerController, AdminController	Handles HTTP requests, controls page flow, receives form submissions, and returns Thymeleaf views.
com.service	BookstoreService, CurrentUserService	Coordinates business logic such as checkout, current user lookup, cart processing, and reservation handling.
com.repository	CatalogRepository, OrderRepository, UserRepository, ReservationRepository	Contains database access logic using JdbcTemplate, including SQL queries and update operations.
com.model	AppUser, Book, Category, Coupon, Reservation	Represents core domain objects used by the system.
com.dto	CartLine, PageResult	Stores view-oriented data used for displaying cart lines, lists, and paginated results.

3.7. API and Route Design

The project is a server-rendered Spring Boot web application. Therefore, it does not expose a large standalone JSON API. Instead, it uses page routes and form submission routes. The route design still follows basic HTTP semantics: GET requests are used to display pages and retrieve information, while POST requests are used for state-changing actions such as registration, adding books to the cart, checkout, profile updates, reservation submission, and manager or admin updates.

Route	Method	Purpose
/	GET	Displays the public home page.
/catalog	GET	Displays the searchable and filterable book catalog.
/catalog/{id}	GET	Displays the detail page of a selected book.
/login	GET	Displays the login page.
/register	GET/POST	Displays the registration form and handles account creation.
/cart	GET	Displays the customer's shopping cart.
/cart/add/{bookId}	POST	Adds a selected book to the cart.
/cart/checkout	POST	Creates an order from the current cart.
/orders	GET	Displays the customer's order history.
/reservations	GET/POST	Displays the reservation page and handles reservation submission.
/profile	GET/POST	Displays and updates user profile information.

/manager/books	GET/POST	Allows managers to view, create, and update book records.
/manager/categories	GET/POST	Allows managers to manage book categories.
/manager/authors	GET/POST	Allows managers to manage author information.
/manager/coupons	GET/POST	Allows managers to manage coupon information.
/manager/orders	GET/POST	Allows managers to view and update customer orders.
/manager/reservations	GET/POST	Allows managers to review and update reservation requests.
/admin/users	GET/POST	Allows administrators to manage user accounts and roles.

3.8. Key Business Logic and Security

3.8.1. Checkout Business Logic

The checkout workflow validates the current user, reads cart lines, verifies book stock, checks coupon validity, calculates subtotal and discount, creates a customer_order record, inserts order_item rows, creates a payment record, creates a shipment record, and records inventory transactions. This business logic prevents overselling and preserves historical order data through snapshots.

```

if (book.stockOnHand() < cartQuantity) {
    reject checkout with a user-friendly message;
}
subtotal = sum(quantity * unitPrice);
discount = validateCoupon(couponCode, subtotal);
grandTotal = subtotal - discount;
create order, order items, payment, shipment, and inventory transaction;

```

3.8.2. Reservation Workflow

When a customer reserves a book, the system records the user, book, quantity, status, reservation time, and expiration time. Managers can update reservation status. Notifications allow customers to

see whether their request is waiting, approved, rejected, or otherwise updated. This workflow is important because it handles demand when immediate checkout is not appropriate or stock is limited.

3.8.3. Security Implementation

- **Password security:** BCrypt hashing is configured through Spring Security PasswordEncoder. Plain text passwords are not stored directly in the database.
- **Authentication:** Spring Security handles login and logout through form-based authentication.
- **Email verification:** The registration workflow includes an email verification mechanism. A verification token is generated for a newly registered user and sent through email. The account is marked as verified only after the user opens the verification link. This reduces fake or mistyped email accounts and improves account reliability.
- **Authorization:** Routes are separated by role. Customers can access shopping-related pages, managers can access management pages, and administrators can access user management features.
- **SQL injection prevention:** JdbcTemplate parameter placeholders are used instead of manual string concatenation for database queries.
- **XSS prevention:** Thymeleaf escapes output by default when standard text rendering is used.
- **CSRF protection:** Spring Security CSRF protection is used for state-changing form requests.

3.8.4. Email Verification Workflow

After a customer submits the registration form, the system creates the user account with an unverified email status. A verification token is generated and stored with an expiration time. The system sends a verification link to the registered email address. When the user opens the link, the application validates the token, checks whether it is expired or already used, updates the user's email_verified field, and marks the token as used. This workflow ensures that the registered email belongs to the user and improves account reliability.

3.9. Advanced Features and AI Assistance

3.9.1. Advanced Features

The IU Bookstore Management System includes several advanced features beyond basic book listing and order creation. These features improve system usability, support management tasks, and make the application more practical for real-world bookstore operations.

Advanced Feature	Implementation Evidence	Value
Dashboard with charts	dashboard.html, Chart.js, dashboard-related controller methods	Provides a visual summary of important bookstore data, such as books, orders, users, and sales-related information. This helps managers monitor the system more efficiently.
CSV export	/manager/export/books.csv, manager export function	Allows managers to export book data into a CSV file for reporting, backup, or offline analysis.
Email verification	Verification token workflow, email_verified status, email sending configuration	Ensures that newly registered users confirm ownership of their email address before fully using the system.
Reservation workflow	reservations table, ReservationController, manager reservation update functions	Allows customers to reserve books and enables managers to review, approve, reject, or update reservation requests.
Coupon management	coupons table, manager coupon management routes	Supports discount codes during checkout and allows managers to control promotional offers.
Role-based access control	roles, user_roles, SecurityConfig	Separates access rights between customers, managers, and administrators. This improves system security and prevents unauthorized access to management features.
Review system	reviews table, book detail page, review-related functions	Allows customers to provide ratings and feedback for books. This improves user interaction and supports customer decision-making.

The advanced features are designed to support both customers and system managers. Customers can search books, add books to cart, place orders, reserve books, apply coupons, and write reviews. Managers can maintain book information, manage authors and categories, process reservations, manage orders, and export book data when needed. Administrators can manage user accounts and roles.

3.9.2. AI Assistance Report

AI tools were used as learning and productivity aids during the development and documentation of the IU Bookstore Management System. The team used AI to explain Spring Boot errors, review database design ideas, improve report wording, and compare the project against the final project requirements. However, AI-generated suggestions were not used blindly. The team reviewed and adjusted all suggestions based on the actual source code, ERD, route structure, and database schema of the project.

Component	AI Support	Human Modification and Learning
Database design	AI was used to review whether the ERD followed relational design principles and whether many-to-many relationships were handled properly.	The team checked the actual ERD and confirmed relationships such as user_roles, book_authors, and book_categories. Incorrect claims about missing or unsupported tables were removed from the report.
Normalization analysis	AI helped explain 1NF, 2NF, 3NF, junction tables, and foreign key relationships.	The team rewrote the normalization section to match the actual ERD instead of using generic database descriptions.
Authentication and authorization	AI was used to explain Spring Security, role-based access control, login, logout, and route protection.	The team checked the actual SecurityConfig and documented how customers, managers, and administrators access different parts of the system.
Business logic	AI helped explain checkout, reservation, coupon validation, and order processing in academic language.	The team removed unsupported features such as shipment tracking, notification center, inventory transaction, and audit log if they were not present in the ERD or source code.

Report structure	AI helped organize the report into sections such as ERD, database creation, normalization, application structure, route design, business logic, security, advanced features, and conclusion.	The team adapted the structure to match the project requirements and the actual implementation.
Debugging support	AI was used to understand Maven, MySQL, Spring Boot, Git, and route-related errors during development.	The team manually applied fixes, tested the project locally, and verified results through the terminal, browser, and GitHub workflow.

CHAPTER 4 IMPLEMENTATION, TESTING AND DEPLOYMENT

4.1. Implementation Process

Requirement	Implementation Evidence	Status
User registration, login, and logout	AuthController, SecurityConfig, UserRepository, email verification workflow	Implemented
Role-based user access	ADMIN, MANAGER, STAFF, CUSTOMER roles	Implemented
Role-based access control	SecurityConfig route protection for /admin/**, /manager/**, and customer routes	Implemented
Profile management	ProfileController and profile templates	Implemented
CRUD for main entities	ManagerController, CatalogRepository, ReservationRepository, OrderRepository	Mostly implemented
Advanced search, filtering, and pagination	CatalogRepository search logic and PageResult	Implemented mainly for books
Many-to-many relationships	user_role, role_permission, book_author	Implemented
Business logic	BookstoreService checkout workflow, stock validation, coupon validation	Implemented
Soft delete	Book status-based deletion	Implemented
Audit trail	audit_log table and admin audit page	Implemented
CSV export	/manager/export/books.csv	Implemented
Notification center	notification table and notification routes	Implemented
Email verification	Email verification token, email_verified status, and verification link	Implemented
Deployment	Docker Compose local setup and public ngrok URL for live demonstration	Implemented for demo

Automated tests	src/test not detected	Pending
-----------------	-----------------------	---------

Table 4.1: Summary of Requirement Implementation Status

4.1.1. Phase 1 - Database Implementation

The first phase focused on creating schema.sql with normalized tables, constraints, foreign keys, check constraints, and indexes. The schema supports user roles, catalog management, cart/order workflows, reservation workflows, notifications, audit logging, and inventory history.

DatabaseSeeder.java was then used to insert demo users, roles, permissions, categories, authors, books, coupons, a reservation, a notification, and an initial audit log.

4.1.2. Phase 2 - Backend Development

The backend was implemented through Spring Boot controllers, services, repositories, models, and DTOs. Controllers define routes and user flows. Services handle current-user and checkout logic. Repositories use JdbcTemplate for SQL operations. Models represent system entities, while DTOs such as CartLine and PageResult support view rendering.

4.1.3. Phase 3 - Frontend Integration

The frontend was integrated directly inside the Spring Boot module because the project uses Thymeleaf server rendering. Templates implement home, login, register, catalog, cart, orders, reservations, profile, notifications, dashboards, and management pages. Static CSS provides the consistent visual design, while JavaScript supports interactive elements and charts. This integrated structure is acceptable because Thymeleaf is a recognized hybrid server-rendered frontend option.

4.1.4. Phase 4 - Advanced Features and Polishing

The later phase focused on advanced requirements: dashboard charts, notification center, CSV export, role-based UI navigation, improved login/register interface, reservation display, and audit log review. Git commits show iterative improvements and branch integration into dev.

4.2. User Guide and Feature Walkthrough

The main user interface is organized around role-specific navigation. Customers interact with public catalog pages, cart, checkout, orders, reservations, notifications, and profile. Managers and staff receive management pages for catalog, coupons, orders, reservations, dashboard and export. Administrators receive user management and audit pages.

Demo Test Accounts:

- Admin: admin@test.com / Admin123!
- Manager: manager@test.com / Manager123!
- Staff: staff@test.com / Staff123!
- Customer: customer@test.com / Customer123!

4.2.1. Getting Started

A new user begins from the public home page or catalog page. Public visitors can browse available books without logging in, but actions such as adding books to cart, checking out, viewing orders, updating a profile, or creating reservations require authentication. The login page accepts email and password. The register page collects customer information and validates required fields such as name,

email, phone, password, and confirmation password. After registration, a customer can log in and use the customer functions.

1. Open the application URL in a browser.
2. Use Register to create a customer account, or use one of the prepared test accounts.
3. Log in through the login form.
4. Use the navigation menu to access catalog, cart, orders, reservations, notifications, and profile.
5. Log out after completing the demo to verify session handling.

4.2.2. Customer Feature Walkthrough

The customer workflow begins with catalog browsing. A customer searches by keyword, filters by category or stock status, opens a book detail page, adds the book to cart, reviews cart quantities, optionally enters a coupon, and submits checkout. During checkout, the application validates the stock level and calculates the final total. If the order is accepted, the system creates an order summary and the customer can review it under the orders page. If a desired book is unavailable or requires staff follow-up, the customer can submit a reservation request and later check reservation status or notifications.

Customer action	Page / route	Expected result
Browse books	/catalog	Books are displayed with search, filter, sort, and pagination options.
Add to cart	/cart/add/{bookId}	Selected book appears in the cart with quantity and price snapshot.
Checkout	/cart/checkout	Order, order item, payment, shipment, and inventory records are created if stock is valid.
Reserve book	/reservations/book/{bookId}	Reservation row is created with WAITING or relevant status.
Read notification	/notifications	Unread message can be marked as read.
Update profile	/profile	Customer information or password is updated after validation.

4.2.3. Manager, Staff, and Admin Walkthrough

Managers focus on catalog and operational control. Staff users support order and reservation follow-up according to the access permissions configured in SecurityConfig. Administrators focus on governance, user management, and audit log review. They can manage the catalog, categories, authors, coupons, order status, reservation status, dashboard metrics, and CSV export. Administrators focus on governance: they can view and manage user records and inspect audit logs. During the live demo, the team should show how the same application changes available functions after logging in with different accounts. This is important evidence for the role-based access control requirement.

Role	Primary pages	Demo points
Manager	/manager/books, /manager/categories, /manager/authors, /manager/coupons	Create/update book data, soft delete, manage lookup data and discounts.
Staff	/manager/orders, /manager/reservations	Update operational status and support customer requests.
Admin	/admin/users, /admin/audit	Create/deactivate users, change roles where implemented, and review audit events.

Figure 4.1: Registration Interfaces

Figure 4.2: Book Catalog Interfaces

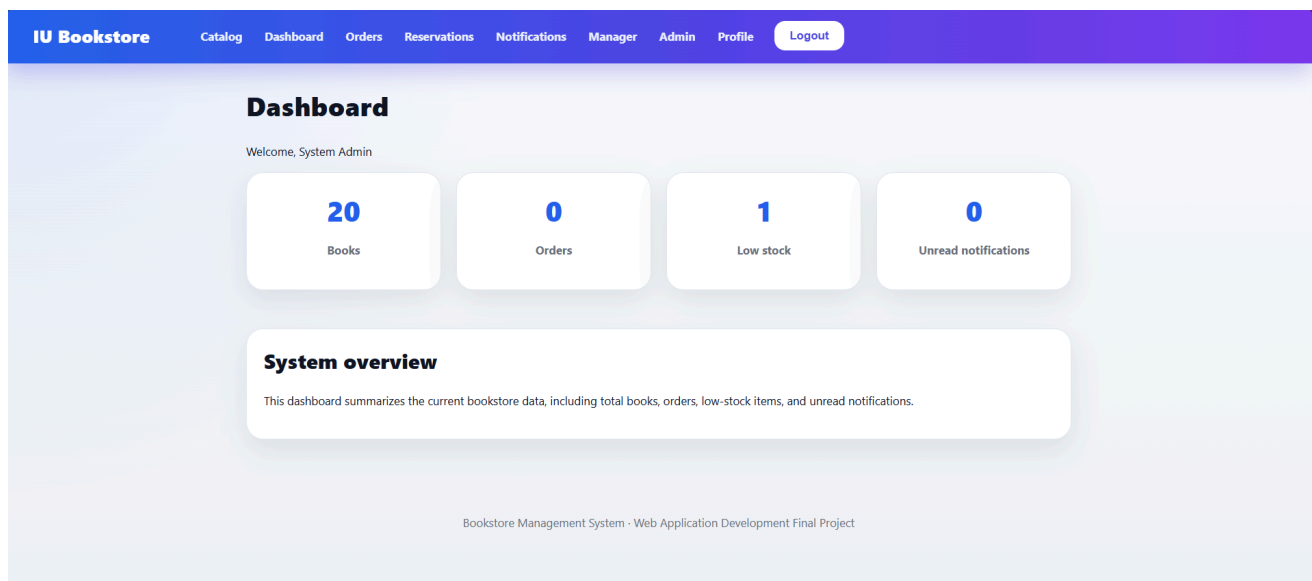


Figure 4.3: Manager Dashboard

4.3. Testing and Quality Assurance

Testing was planned around requirement coverage rather than only happy-path clicking. Because the uploaded repository does not yet include a src/test folder, this report records manual testing scenarios and recommends adding automated unit/integration tests before final grading. The most important tests are authentication, RBAC, CRUD behavior, checkout stock validation, coupon validation, reservation updates, notifications, CSV export, and database integrity.

The recommended testing strategy has three levels. Unit tests should check pure business logic such as coupon calculation, stock validation, and password/profile validation. Repository or integration tests should run against H2 or a test MySQL database to verify that SQL queries return expected records and respect constraints. Controller/security tests should verify that routes return expected status codes and that customers cannot access manager or administrator functions. Manual browser testing should then verify the complete user experience, visual layout, and demo flow.

No.	Test scenario	Expected result	Status
1	Register with valid customer data	Account is created and can log in	Manual test required
2	Register with duplicate email	System rejects duplicate email with clear message	Manual/automated test required
3	Login with correct credentials	User is redirected to role-appropriate page	Manual test required
4	Login with wrong password	Login fails without exposing sensitive information	Manual test required
5	Customer accesses /admin/users	Access is denied or redirected	Manual/automated security test required
6	Search catalog by keyword	Matching books are shown with pagination	Manual test required

7	Filter books by category/status/price/stock	Filtered results match criteria	Manual test required
8	Manager creates a book	Book appears in catalog and manager list	Manual test required
9	Manager soft deletes a book	Book status changes without removing historical orders	Manual test required
10	Checkout with enough stock	Order, order items, payment, shipment, and inventory transaction are created	Manual/integration test required
11	Checkout with insufficient stock	Checkout is blocked with user-friendly error	Manual/integration test required
12	Apply valid coupon	Discount is calculated correctly	Manual/unit test required
13	Submit reservation	Reservation is stored and visible to customer/manager	Manual test required
14	Manager updates reservation status	Customer notification is created or updated	Manual test required
15	Export books CSV	CSV downloads with expected book columns	Manual test required

Table 4.2: Testing Matrix

4.4. Bug Tracking

Issue	Cause	Solution / recommendation
MySQL connection errors	Local database port or credentials differed across machines	Use environment variables and document DB_URL, DB_USER, DB_PASSWORD in README and .env.example.
Schema initialization confusion	spring.sql.init.mode was disabled for MySQL	Document when to run schema.sql manually and when DatabaseSeeder runs.
Frontend/backend folder confusion	Thymeleaf pages are inside backend resources	Explain server-rendered frontend structure in report and README.
Missing automated tests	Development focused on feature completion first	Add minimum tests for auth, repository queries, checkout service, and controller access.
Hardcoded password in application-mysql.properties	Local development convenience	Replace with \${DB_PASSWORD} before pushing final version.

4.5. Deployment Process

The final project requirement expects a public URL that works outside localhost. For the final demonstration, the application is exposed through a public ngrok URL:

<https://boned-gerbil-tattle.ngrok-free.dev>. The repository also contains docker-compose.yml for local MySQL support and DEPLOYMENT.md documentation. This deployment approach is suitable for classroom demonstration because it allows the instructor to access the locally running Spring Boot application through a public network. A recommended deployment process is listed below. However, the ngrok URL is a temporary tunnel rather than a permanent production deployment. Therefore, before the live demo, the team must keep the Spring Boot application running, keep the ngrok terminal active, confirm that app.base-url matches the current public URL, and test the registration, email verification, login, catalog, cart, reservation, and manager dashboard flows through the public link.

Local execution is useful for development, but it is not sufficient for grading because the instructor must be able to access the application from a public network. The deployment environment should separate source code from configuration. Database URL, username, password, active Spring profile, and any future API keys must be stored as environment variables rather than committed into Git. The final submission should include the public URL, test accounts, and a short uptime check statement.

6. Create production database on Railway, Render, or another supported MySQL/PostgreSQL provider.
7. Run schema.sql on the production database, then prepare demo records either through seed SQL or DatabaseSeeder.
8. Move database credentials to environment variables: DB_URL, DB_USER, DB_PASSWORD, and DB_DRIVER.
9. Update application configuration to read environment variables instead of hardcoded local credentials.
10. Build the application with Maven: ./mvnw clean package.
11. Deploy the generated jar to Render, Railway, Fly.io, or a similar Java-supported platform.
12. Verify public routes, login accounts, dashboard, catalog, checkout, reservation, and manager/admin pages.
13. Insert the final public URL into the README, report cover page, and presentation slides.

Environment variable	Example purpose
SPRING_PROFILES_ACTIVE	Use mysql or production profile.
DB_URL	Production JDBC URL.
DB_USER	Production database username.
DB_PASSWORD	Production database password stored outside Git.
SERVER_PORT	Hosting platform assigned port if required.
LOG_LEVEL	Production log level such as INFO or WARN.

Table 4.3: Deployment Environment Variables

CI/CD is optional for this academic submission, but a simple pipeline is recommended: push to dev, run build and tests, merge to main, deploy from main, and verify the live URL. If automated deployment is not available, the team should still document the manual deployment commands and keep a backup demo video or screenshots in case internet access fails during presentation.

Role	Email	Password
Admin	admin@test.com	Admin123!
Manager	manager@test.com	Manager123!
Staff	staff@test.com	Staff123!
Customer	customer@test.com	Customer123!

Table 4.4: Test Accounts

4.6. Results and Analysis

The implementation achieves the major functional goals of a bookstore management system. The database design exceeds the minimum complexity requirement by providing 25 tables and 178 columns. The application includes four user roles, role-based navigation, authentication, customer catalog browsing, cart/checkout, reservation, profile, order history, manager catalog management, dashboard reporting, CSV export, notification center, and audit log review. These features demonstrate full-stack integration rather than isolated database scripts or static pages.

The strongest technical result is the relational schema, which contains multiple relationship types, junction tables, audit/event tables, and indexes. Another important result is that the project uses Spring Security and BCrypt, which directly address the non-negotiable security requirements. The main improvement areas are deployment evidence, automated tests, README completeness, and removal of hardcoded credentials.

4.7. Challenges and Solutions

Challenge	Explanation	Solution
Database complexity	Bookstore data includes users, roles, books, authors, orders, payments, reservations, notifications, and audit logs.	The team separated independent concepts into different tables and used junction tables for many-to-many relationships.
Role-based access	Different users should see different features.	Spring Security was configured and route groups were separated into customer, manager/staff, and admin areas.
Preventing overselling	Customers should not check out more books than available stock.	Checkout workflow checks stock before creating orders and records inventory transactions.
UI consistency	Many templates can become visually inconsistent.	Shared CSS and reusable Thymeleaf layout ideas were applied; final polish should continue before demo.
Git collaboration	Multiple feature branches and commits need to be integrated safely.	dev branch integration and meaningful commit messages were used; final merges should be reviewed before main submission.

CHAPTER 5 CONCLUSION AND FUTURE WORK

5.1. Summary of Key Findings

The IU Bookstore Management System demonstrates a realistic web application for university bookstore operations. It integrates user authentication, RBAC, catalog management, search and filtering, cart and checkout workflow, reservation handling, notifications, dashboard metrics, CSV export, audit logging, and a normalized relational database. The schema is significantly more complex than the minimum requirement and includes strong relational modeling practices.

5.2. Reflection on Objectives

The project successfully applies Web Application Development concepts from frontend pages and HTTP routing to backend service logic and database integration. The team learned how Spring Boot organizes a project, how Spring Security protects routes, how JdbcTemplate supports parameterized database access, how Thymeleaf renders server-side views, and how relational database constraints protect data integrity. The project also strengthened teamwork through Git branches, commits, feature integration, and documentation.

5.3. Future Work and Recommendations

- Replace the temporary ngrok demonstration URL with a permanent cloud deployment on a platform such as Render, Railway, or Fly.io.
- Add automated tests for repository queries, service logic, controller access control, registration, checkout, and reservation status updates.
- Integrate email reminders for reservations, order confirmations, and low-stock alerts.
- Add VNPay or Stripe payment gateway integration for real payment confirmation.
- Improve dashboard analytics with date ranges, revenue trends, best-selling books, and stock forecasting.
- Add image upload and management for book covers with file validation and storage control.
- Harden production security with HTTPS, rate limiting, secure cookies, CSP headers, and secret management.
- Expand the README and API documentation so another developer can run the system without asking the team.

REFERENCES

- [1] Spring Boot Documentation. Spring Boot Reference Guide.
- [2] Spring Security Documentation. Authentication, Authorization, Password Encoding, and CSRF Protection.
- [3] Thymeleaf Documentation. Server-side Java Template Engine for Web Applications.
- [4] MySQL Documentation. SQL Constraints, Indexing, and Foreign Keys.
- [5] Final Project - Web Application Development Course: Project Description & Requirements.
- [6] IU Bookstore Management System source repository, schema.sql, Java classes, templates, and documentation files.

APPENDICES

Appendix A. Routes and Test Accounts

Route group	Routes	
Public	/, /catalog, /catalog/{id}, /login, /register	
Customer	/cart, /orders, /reservations, /notifications, /profile	
Staff	dashboard and operational pages according to the final SecurityConfig configuration	
Manager/Admin	/manager/books, /manager/categories, /manager/authors, /manager/coupons, /manager/orders, /manager/reservations, /manager/export/books.csv	
Admin	/admin/users, /admin/audit	
Role	Email	Password
Admin	admin@test.com	Admin123!
Manager	manager@test.com	Manager123!
Staff	staff@test.com	Staff123!
Customer	customer@test.com	Customer123!

Appendix B. Database Tables

Table	Columns
app user	9
role	3
user role	3
permission	3
role permission	3
address	11
category	7
author	8
book	18
book author	3
coupon	9
promotion	7
cart	4
cart item	5
wishlist	5
wishlist item	3
reservation	8
customer order	13
order item	7
payment	8

email verification token	6
inventory transaction	9
shipment	7
review	7
notification	10
audit log	8

Appendix C. Git and Repository Checklist

Requirement	Evidence / status
Minimum 50 commits	More than 80 commits detected in Git history
Meaningful commit messages	Examples include feat(ui), feat(security), chore(config), doc, fix(config)
.gitignore	Present
README	Present but must be expanded
database/schema.sql	Present
database/seed-data.sql	Present but currently nearly empty
ERD file	database/ERD1.svg present
docs/API.md and docs/DEPLOYMENT.md	Present
LICENSE	Not detected; should be added before final submission
Automated tests	Not detected; should be added before final submission

Appendix D. Requirement Compliance Checklist

Requirement area	Status
User management	Satisfied: registration, login/logout, roles, profile management, admin user management, BCrypt password hashing, and email verification are implemented.
CRUD	Mostly satisfied for catalog and management entities.
Search/filter/pagination	Satisfied mainly for catalog/book listing.
Complex relationships	Satisfied: 25 tables, junction tables, foreign keys.
Business logic	Satisfied: stock, coupon, checkout, reservation, notifications, audit.
Advanced features	Satisfied: dashboard charts, CSV export, notifications, audit log.
Security	Mostly satisfied: BCrypt, RBAC, CSRF, parameterized SQL; production hardening recommended.
Deployment	Satisfied for demo through public ngrok URL; permanent hosting is recommended for production.
Report	This document follows the required sections while using a PDMS-style chapter layout.